Feature Engineering

> Part of the Data Science / ML curriculum. This document covers turning cleaned panel/time-series data into model-ready features: time-based, lag, rolling, growth, and ratio features, plus scaling and categorical encoding.

1. What feature engineering is, and why it matters

Feature engineering is deriving new columns from raw data that make the underlying signal easier for a model to use. A model can only learn from the columns you give it — no algorithm can invent a useful signal (like "this quantity tends to persist from one period to the next") out of thin air if the only column available is the raw, untransformed value. In many practical projects, feature engineering — not model choice — is what separates a mediocre model from a good one.

2. Feature types for panel/time-series data

For data with both an entity dimension (e.g., one row per store/customer/region per period) and a time dimension, a specific family of features recurs constantly:

Feature type	What it is	pandas pattern
Time index	A numeric count of periods elapsed since some reference point	`df["t"] = df["year"] - REFERENCE_YEAR`
Rolling window statistic	A smoothed value over the last *N* periods (mean, std, min, max)	`df.groupby("entity")["value"].transform(lambda s: s.rolling(5).mean())`
Lag feature	The value from *N* periods ago, for the same entity	`df.groupby("entity")["value"].shift(n)`
Growth / rate-of-change	Absolute or percentage change from the previous period	`df.groupby("entity")["value"].diff()` / `.pct_change()`
Ratio / intensity	One quantity normalized by another	`df["intensity"] = df["numerator"] / df["denominator"]`

2.1 Time index

```
df["t"] = df["year"] - 1990
```

The raw year (e.g., `2019`) usually isn't a good regression input directly — its absolute scale carries no meaning to a linear model (there's no reason `2019` should mean something 1.5× as large as `1346`). A normalized index counting periods from a reference point gives the model a clean, meaningful numeric trend variable instead.

2.2 Rolling window statistics

```
df["value_5yr_avg"] = df.groupby("entity")["value"].transform(lambda s: s.rolling(5).mean())
```

`groupby(...).transform(...)` is the key pattern here: it computes the rolling statistic *within each entity's own history* and returns a result aligned back to the original DataFrame's index (unlike `groupby(...).apply(...)`, which can return a differently-shaped result). `rolling(5)` means "the trailing window of 5 rows ending at this row" — by default this needs 5 rows of history to produce a value, so the first 4 rows of each entity's series will be `NaN` (see §3 for why this is correct, not a bug). `.rolling()` also supports `.std()`, `.min()`, `.max()`, `.sum()`, and a `min_periods` argument if you want a value before the window is completely full.

2.3 Lag features

```
df["value_lag1"] = df.groupby("entity")["value"].shift(1)   # previous period's value
df["value_lag2"] = df.groupby("entity")["value"].shift(2)
```

Why lag features work — temporal autocorrelation. A lag feature is useful because many real-world quantities change slowly: this period's value is highly correlated with last period's, because whatever process generates the data (economic activity, physical infrastructure, biological/natural systems) has *inertia* — it doesn't jump discontinuously. Giving a model direct access to recent history lets it exploit that inertia without needing to understand *why* the series behaves that way.

The stationarity assumption, and when it breaks. Using lag/rolling features implicitly assumes the relationship between past and future values is stable over time — the same correlation structure that held during training continues to hold when the model is used. This is reasonable for slowly-evolving series but breaks during structural shocks: a sudden shift in the underlying process (a policy change, a crisis, a regime change) the model has never seen a lagged example of. Recognizing where a model is likely to violate this assumption is as important as building the features in the first place — see Time-Series Forecasting, §6 for a more formal treatment of stationarity and how to actually test for it.

2.4 Growth / rate-of-change features

```
df["value_change"]     = df.groupby("entity")["value"].diff()          # absolute change
df["value_pct_change"] = df.groupby("entity")["value"].pct_change() * 100   # percentage change
```

`.diff()` and `.pct_change()` both default to a 1-period lag but accept a `periods=` argument for a different lag distance. These are useful both as model features and as an EDA/reporting metric in their own right (e.g., "which segments grew fastest").

2.5 Ratio / intensity features

```
df["intensity"] = df["numerator"] / df["denominator"]
```

A ratio feature normalizes one quantity by another, making values comparable across entities of very different scale (e.g., a per-capita or per-unit measure). **Check both the numerator and denominator's missingness**

before relying on a ratio feature — a ratio is undefined wherever either input is missing or the denominator is zero, and ratio features often have meaningfully *higher* missingness than either of their inputs alone.

3. Structural vs. genuine missingness

After adding lag/rolling features, the first few rows of each entity's series will have `NaN` values *by construction* — a 5-period rolling mean needs 5 periods of history, so an entity's first 4 rows can't have one yet. **This is correct and should not be filled or imputed.** Silently filling these with 0 or an interpolated guess would inject fabricated signal into rows where none actually exists.

Contrast this with **genuine missingness** — a value that's absent because the underlying data was never collected (e.g., a normalizing denominator missing for some entities, as in §2.5). Genuine missingness usually means treating that feature as **optional**: report results both with and without it, or exclude it from the primary feature set, rather than dropping rows or filling with an assumed value — either of those would distort the training set specifically for the entities/periods affected, which is rarely a neutral choice.

4. Scaling and normalization

Some models (notably linear models with regularization, and anything based on distance calculations) are sensitive to the *scale* of input features — a feature ranging in the thousands can dominate a feature ranging from 0 to 1 purely because of units, not because it's more informative. Tree-based models (Random Forest, gradient boosting) are **not** sensitive to feature scale, since they split on thresholds rather than computing distances or weighted sums directly — but it's still good practice to know the tools:

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler

scaler = StandardScaler()               # transforms to mean 0, standard deviation 1
X_train_scaled = scaler.fit_transform(X_train)   # fit on training data only
X_test_scaled  = scaler.transform(X_test)        # reuse the same fitted scaler on test data

# MinMaxScaler instead rescales to a fixed range (default 0–1):
minmax = MinMaxScaler()
X_train_scaled = minmax.fit_transform(X_train)
```

Always `fit` a scaler on training data only, then `transform` (not re-`fit`) the test data. Fitting on the combined train+test set (or on test data alone) leaks information about the test set's distribution into preprocessing — the same category of mistake as fitting a categorical encoder on the wrong subset (§5).

5. Encoding categorical features

Models need numeric input — a categorical column (a set of discrete labels) needs to be converted to numbers before it can be used as a feature.

5.1 Label encoding

```
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()
encoder.fit(ALL_KNOWN_CATEGORIES)     # fit ONCE, on the complete, known list of categories
df["category_encoded"] = encoder.transform(df["category"])
```

This assigns each category an arbitrary integer code. **Fit the encoder once, on the complete set of possible categories, and reuse the same fitted encoder everywhere** (training data, test data, and any future data) — never refit it on a subset. `LabelEncoder.fit()` assigns integer codes based on whatever list it's given, sorted; if you fit it again on a smaller list (e.g., just the test set, or just one category), the same category can silently get a *different* integer than it had during training, corrupting whatever a model learned about that feature. This is a two-line function call but a very easy mistake to make and a subtle one to debug — always keep one fitted encoder object and pass it around, never call `.fit()` a second time.

A label-encoded integer implies an ordering that usually isn't meaningful (category `3` isn't "more" than category `1` in any real sense) — this is fine for tree-based models (which split on thresholds and don't assume any linear relationship between the code and the target), but can mislead a linear model into fitting a spurious linear relationship to what is really just an arbitrary numbering.

5.2 One-hot encoding

```
from sklearn.preprocessing import OneHotEncoder
import pandas as pd

# scikit-learn's encoder:
encoder = OneHotEncoder(sparse_output=False, handle_unknown="ignore")
encoded = encoder.fit_transform(df[["category"]])

# the pandas shortcut for the same idea:
df_encoded = pd.get_dummies(df, columns=["category"])
```

One-hot encoding creates one binary (0/1) column per category, avoiding the false-ordering problem of label encoding — appropriate for linear models and for any categorical feature where the categories genuinely have no natural order. The trade-off is that it adds one column per distinct category, which becomes impractical for a feature with very many categories (a "high-cardinality" categorical column) — label encoding, or a technique designed for high cardinality (e.g., target encoding), is a better fit in that case. `handle_unknown="ignore"` matters in production: it tells the encoder what to do if it sees a category at prediction time it never saw during fitting, rather than raising an error.

6. Core libraries summary

Library	Used for
pandas	`groupby` / `transform` / `shift` / `rolling` / `diff` / `pct_change` — all the feature-construction operations above
NumPy	underlying numeric operations
scikit-learn	`StandardScaler`, `MinMaxScaler`, `LabelEncoder`, `OneHotEncoder`

7. Testing feature engineering code

Once feature-construction logic is extracted from an exploratory notebook into reusable functions, it becomes straightforward — and worthwhile — to unit-test with `pytest`, using a small, hand-built synthetic DataFrame rather than real production data:

```
import pandas as pd

def add_lag_feature(df: pd.DataFrame, column: str, periods: int = 1) -> pd.DataFrame:
    df = df.copy()
    df[f"{column}_lag{periods}"] = df.groupby("entity")[column].shift(periods)
    return df

def test_lag_feature_is_nan_for_first_row_per_entity():
    df = pd.DataFrame({
        "entity": ["A", "A", "B", "B"],
        "value":  [10,   20,  100,  110],
    })
    result = add_lag_feature(df, "value", periods=1)
    assert result["value_lag1"].isna().tolist() == [True, False, True, False]
    assert result.loc[1, "value_lag1"] == 10   # A's second row lags A's first row
    assert result.loc[3, "value_lag1"] == 100  # B's second row lags B's first row, not A's
```

This kind of test deliberately includes **two entities**, so it would catch a mistake where a lag/rolling operation is accidentally computed across the whole DataFrame instead of per-entity (a very common bug: forgetting the `.groupby("entity")` and letting one entity's last row "leak" into another entity's first row).

See also

- Data Science / ML index
- EDA & Data Engineering — the previous step, cleaning the raw data these features are built from
- Regression Models — the next step, using these features to train models